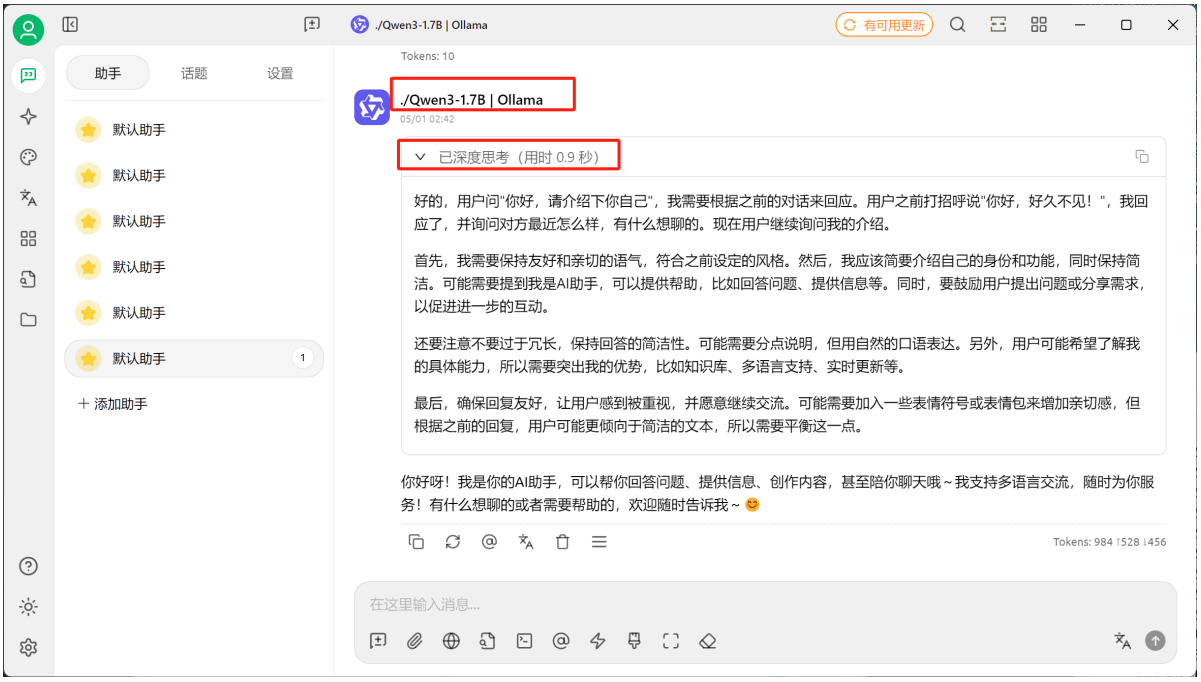
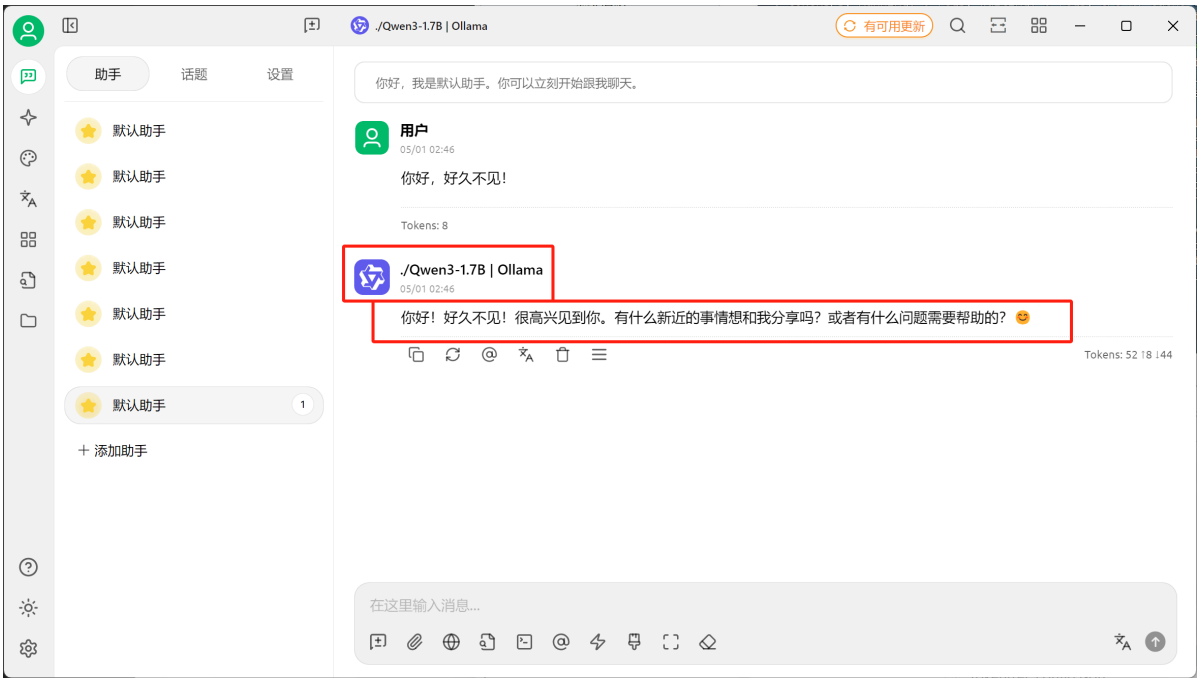
自由控制Qwen3模型的思考模式

- **vLLM开启思考模式对话效果展示（同SGLang）**





- **vLLM关闭思考模式对话效果展示（同SGLang）**



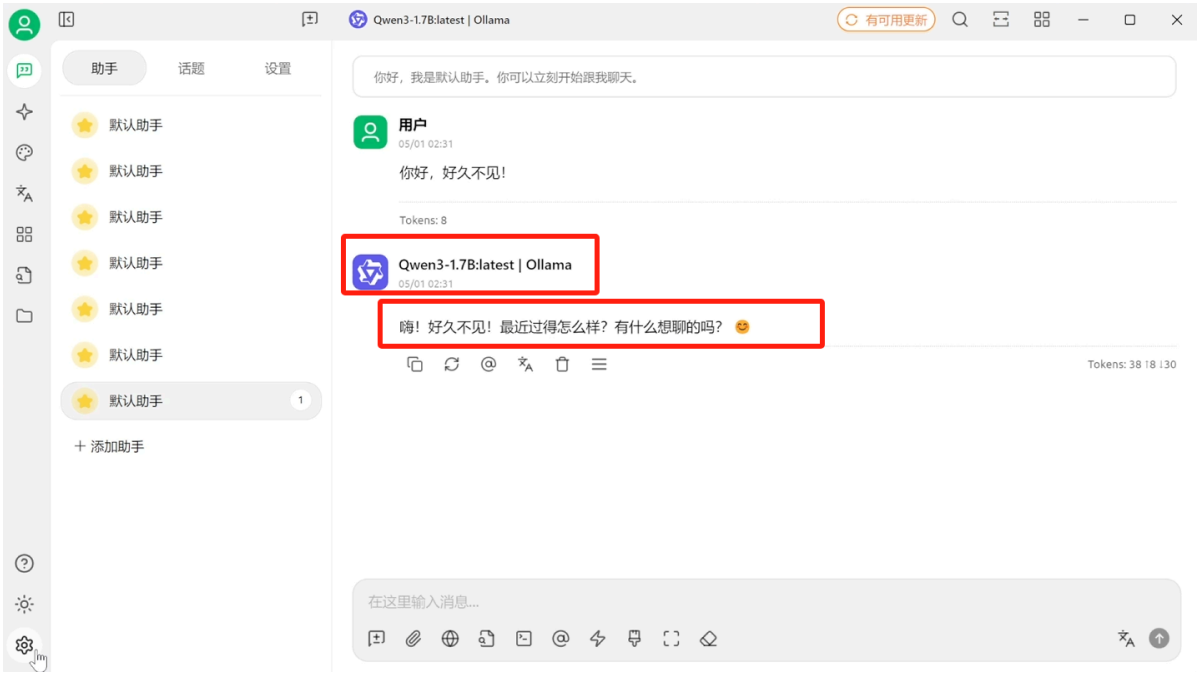


- **ollama开启思考模式对话效果展示**





- **ollama关闭思考模式对话效果展示**


Qwen3-1.7B:latest | Ollama
有可用更新
助手
话题
设置
默认助手
默认助手
默认助手
默认助手
默认助手
默认助手
默认助手
1
添加助手
你好，我是默认助手。你可以立刻开始跟我聊天。
用户
05/01 02:31
你好，好久不见！
Tokens: 8
Qwen3-1.7B:latest | Ollama
05/01 02:31
嗨！好久不见！最近过得怎么样？有什么想聊的吗？
Tokens: 38 ↑8 ↓30
在这里输入消息...



- 课件领取：



1. Qwen3模型思考模式切换基本方法

作为混合推理模型，能够自由的开启和关闭思考模式至关重要，根据官方说明，Qwen3系列模型默认是开启思考模式的，而在某些时候确实不需要模型进行思考，此时有两种方法能够关闭模型思考方式：

- 其一是“硬关闭”方法，通过一些模型调用框架的参数设置来开启或者关闭模型思考，例如如果是使用transformers框架进行模型调用，则可以通过设置enable_thinking=True 或 False来决定是否开启思考，如下则是开启思考：

```python
from modelscope import AutoModelForCausalLM, AutoTokenizer
```

```python
model_name = "./Qwen3-1.7B"
```

```python
# load the tokenizer and the model
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype="auto",
    device_map="auto"
)
```

```
Loading checkpoint shards: 100% 2/2 [00:16<00:00,  6.93s/it]
```

```python
# prepare the model input
prompt = "你好，好久不见！"
messages = [
    {"role": "user", "content": prompt}
]
text = tokenizer.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=True,
    enable_thinking=True # Switches between thinking and non-thinking modes. Default is True
)
model_inputs = tokenizer([text], return_tensors="pt").to(model.device)
```

```python
# conduct text completion
generated_ids = model.generate(
    **model_inputs,
    max_new_tokens=32768
)
output_ids = generated_ids[0][len(model_inputs.input_ids[0]):].tolist()
```

```python
# parsing thinking content
try:
    # rindex finding 151668 (</think>)
    index = len(output_ids) - output_ids[::-1].index(151668)
except ValueError:
    index = 0

thinking_content = tokenizer.decode(output_ids[:index], skip_special_tokens=True).strip("\n")
content = tokenizer.decode(output_ids[index:], skip_special_tokens=True).strip("\n")

print("thinking content:", thinking_content)
print("content:", content)
```

```
thinking content: <think>
好的，用户发来“你好，好久不见！”，我需要回应这个问候。首先，用户可能是在久别重逢，所以我要保持友好和亲切。可能需要确认对方是否安好，或者是否有其他需要帮助的地方。

接下来，我应该考虑用户可能的意图。他们可能只是想打招呼，或者有更深层的需求，比如寻求帮助、分享信息，或者只是想聊天。需要保持开放，让用户感觉被重视。

然后，我要确保回应简洁但有温度。避免使用过于正式或生硬的语言，让对话自然流畅。可能需要加入一些表情符号或亲切的语气词，比如“❤️”或“😊”，但要根据用户的偏好调整。

另外，要注意用户可能的潜在问题。比如，他们可能在寻找建议，或者需要支持，所以回应中可以适当提供帮助，但不过于主动，以免显得打扰。

最后，保持积极向上的态度，鼓励用户继续交流，同时表达对他们的关心。这样既能回应问候，又能促进进一步的互动。
</think>
content: 啊，好久不见！看到你的消息真的好开心呀～最近过得怎么样？有什么想聊的吗？😊 无论是分享日常、讨论感兴趣的话题，还是需要帮忙的地方，我都在这里哦～ 顺便问一下，最近有没有什么特别想做的事情？❤️
```

而enable_thinking=False则是关闭思考

```python
from modelscope import AutoModelForCausalLM, AutoTokenizer

model_name = "./Qwen3-1.7B"

# load the tokenizer and the model
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype="auto",
    device_map="auto"
)
```

```
Loading checkpoint shards: 100% 2/2 [00:16<00:00, 6.93s/it]
```

```python
# prepare the model input
prompt = "你好，好久不见！"
messages = [
    {"role": "user", "content": prompt}
]
text = tokenizer.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=True,
    enable_thinking=False # Switches between thinking and non-thinking modes. Default is True.
)
model_inputs = tokenizer([text], return_tensors="pt").to(model.device)

# conduct text completion
generated_ids = model.generate(
    **model_inputs,
    max_new_tokens=32768
)
output_ids = generated_ids[0][len(model_inputs.input_ids[0]):].tolist()

# parsing thinking content
try:
    # rindex finding 151668 (</think>)
    index = len(output_ids) - output_ids[::-1].index(151668)
except ValueError:
    index = 0

thinking_content = tokenizer.decode(output_ids[:index], skip_special_tokens=True).strip("\n")
content = tokenizer.decode(output_ids[index:], skip_special_tokens=True).strip("\n")

print("thinking content:", thinking_content)
print("content:", content)
```

```
thinking content:
content: 你好！好久不见！很高兴见到你。有什么新近的事情想和我分享吗？或者有什么想聊的？😊
```

类似的vLLM或者SGLang也可以通过参数设置关闭或开启思考

```
CUDA_VISIBLE_DEVICES=0,1 vllm serve ./Qwen3-1.7B --tensor-parallel-size 2
```

```
INFO 04-30 19:11:16 [launcher.py:34] Route: /detokenize, Methods: POST
INFO 04-30 19:11:16 [launcher.py:34] Route: /v1/models, Methods: GET
INFO 04-30 19:11:16 [launcher.py:34] Route: /version, Methods: GET
INFO 04-30 19:11:16 [launcher.py:34] Route: /v1/chat/completions, Methods: POST
INFO 04-30 19:11:16 [launcher.py:34] Route: /v1/completions, Methods: POST
INFO 04-30 19:11:16 [launcher.py:34] Route: /v1/embeddings, Methods: POST
INFO 04-30 19:11:16 [launcher.py:34] Route: /pooling, Methods: POST
INFO 04-30 19:11:16 [launcher.py:34] Route: /score, Methods: POST
INFO 04-30 19:11:16 [launcher.py:34] Route: /v1/score, Methods: POST
INFO 04-30 19:11:16 [launcher.py:34] Route: /v1/audio/transcriptions, Methods: POST
INFO 04-30 19:11:16 [launcher.py:34] Route: /rerank, Methods: POST
INFO 04-30 19:11:16 [launcher.py:34] Route: /v1/rerank, Methods: POST
INFO 04-30 19:11:16 [launcher.py:34] Route: /v2/rerank, Methods: POST
INFO 04-30 19:11:16 [launcher.py:34] Route: /invocations, Methods: POST
INFO:     Started server process [611252]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

通过关键参数设置 `extra_body={"chat_template_kwargs": {"enable_thinking": False}},` 决定模型是否进行思考：

- 调用vLML

```python
from openai import OpenAI

openai_api_key = "EMPTY"
openai_api_base = "http://localhost:8000/v1"

client = OpenAI(
    api_key=openai_api_key,
    base_url=openai_api_base,
)
```

```python
messages = [
    {"role": "user", "content": "你好"}
]
```

- 默认开启思考模式

```python
response = client.chat.completions.create(
    model="./Qwen3-4B",
    messages=messages,
)
print(response.choices[0].message.content)
```

```
<think>
用户发来“你好”，这是一个常见的问候语。首先，我需要回应以保持对话的连贯性。作为AI助手，我应该用友好、专业的语气来回应，同时展现出愿意帮助解决问题的态度。

接下来，我需要考虑用户可能的意图。用户可能只是想开始对话，或者有具体的问题需要解决。因此，我的回应应该既友好又开放，鼓励用户进一步说明他们的需求。

同时，我需要确保语言简洁明了，避免使用过于复杂的词汇或结构，让用户容易理解和回应。此外，保持口语化风格，让对话更加自然流畅。

可能的回应方向包括：表达问候，询问用户是否有具体问题，或者提供帮助的意愿。例如：“你好！很高兴见到你。有什么我可以帮你的吗？”这样的回应既友好又开放，鼓励用户继续对话。

另外，我需要检查是否有需要特别注意的地方，比如用户可能有隐藏的需求或问题，但目前没有足够的信息来判断。因此，保持回应的通用性，同时提供进一步帮助的选项是合适的。

最后，确保回应符合公司的政策和指导原则，保持中立、专业和友好，不涉及任何可能引起误解的内容。
</think>

你好！很高兴见到你。有什么我可以帮你的吗？😊
```

- 设置参数不进行思考

```python
response = client.chat.completions.create(
    model="./Qwen3-4B",
    messages=messages,
    extra_body={"chat_template_kwargs": {"enable_thinking": False}},
)
print(response.choices[0].message.content)
```

```
你好！有什么我可以帮你的吗？😊
```

但ollama暂时没有这种设置。

- 除此之外，还有一种“软关闭”的方法，即无论使用何种框架无论如何设置参数，都可以在输入内容的时候通过添加 `/think` 和 `/no_think` 来逐轮切换模型的思考模式：

```python
# prepare the model input
prompt = "你好，好久不见！/no_think"
messages = [
    {"role": "user", "content": prompt}
]
text = tokenizer.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=True,
    enable_thinking=True # Switches between thinking and non-thinking modes. Default is True.
)
model_inputs = tokenizer([text], return_tensors="pt").to(model.device)

# conduct text completion
generated_ids = model.generate(
    **model_inputs,
    max_new_tokens=32768
)
output_ids = generated_ids[0][len(model_inputs.input_ids[0]):].tolist()

# parsing thinking content
try:
    # rindex finding 151668 (</think>)
    index = len(output_ids) - output_ids[::-1].index(151668)
except ValueError:
    index = 0

thinking_content = tokenizer.decode(output_ids[:index], skip_special_tokens=True).strip("\n")
content = tokenizer.decode(output_ids[index:], skip_special_tokens=True).strip("\n")

print("thinking content:", thinking_content)
print("content:", content)
```

```
thinking content: <think>

</think>
content: 你好！好久不见，很高兴见到你！有什么想聊的吗？😊
```

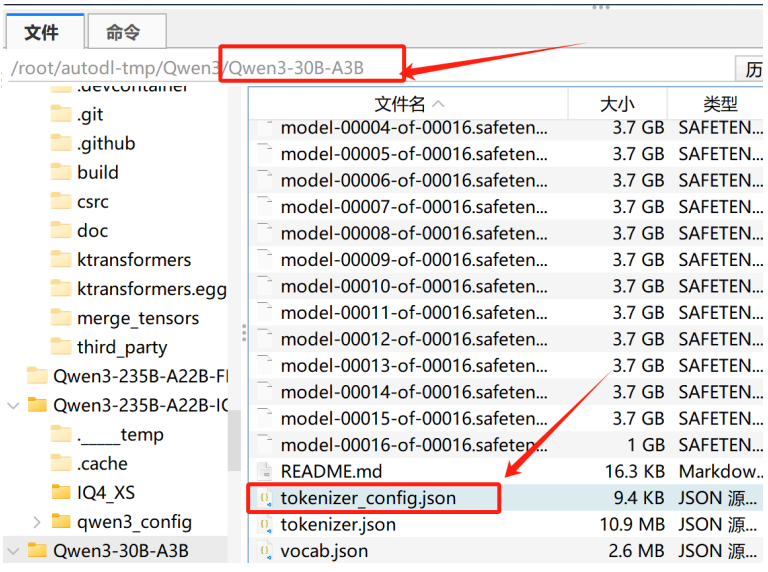
课件领取：



2.Qwen3模型思考方式切换的底层逻辑

而实际上，作为语言模型，影响模型行为的一定是“输入的内容”，因此，哪怕是我们通过一些参数设置，实际上也是影响模型最终的输入，这点我们可以通过解析Qwen3内置提示词模板进行解读：以下是Qwen3内置提示词模板解析：

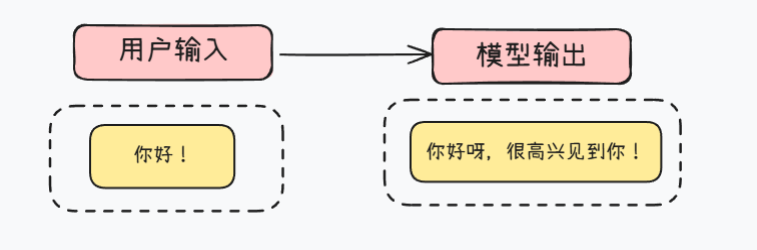
内置提示词模板可以在任意模型的`tokenizer_config.json`中查看：



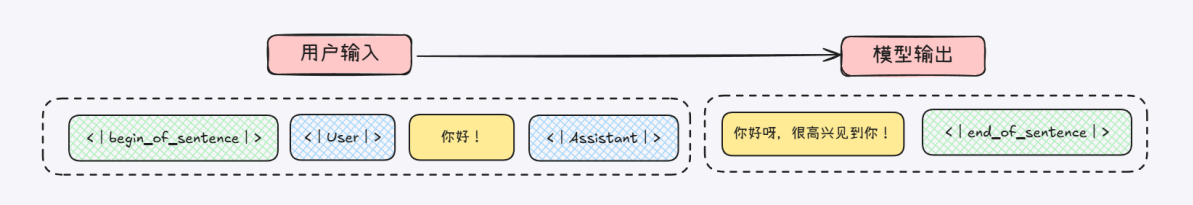
双卡H800 tokenizer_config.json

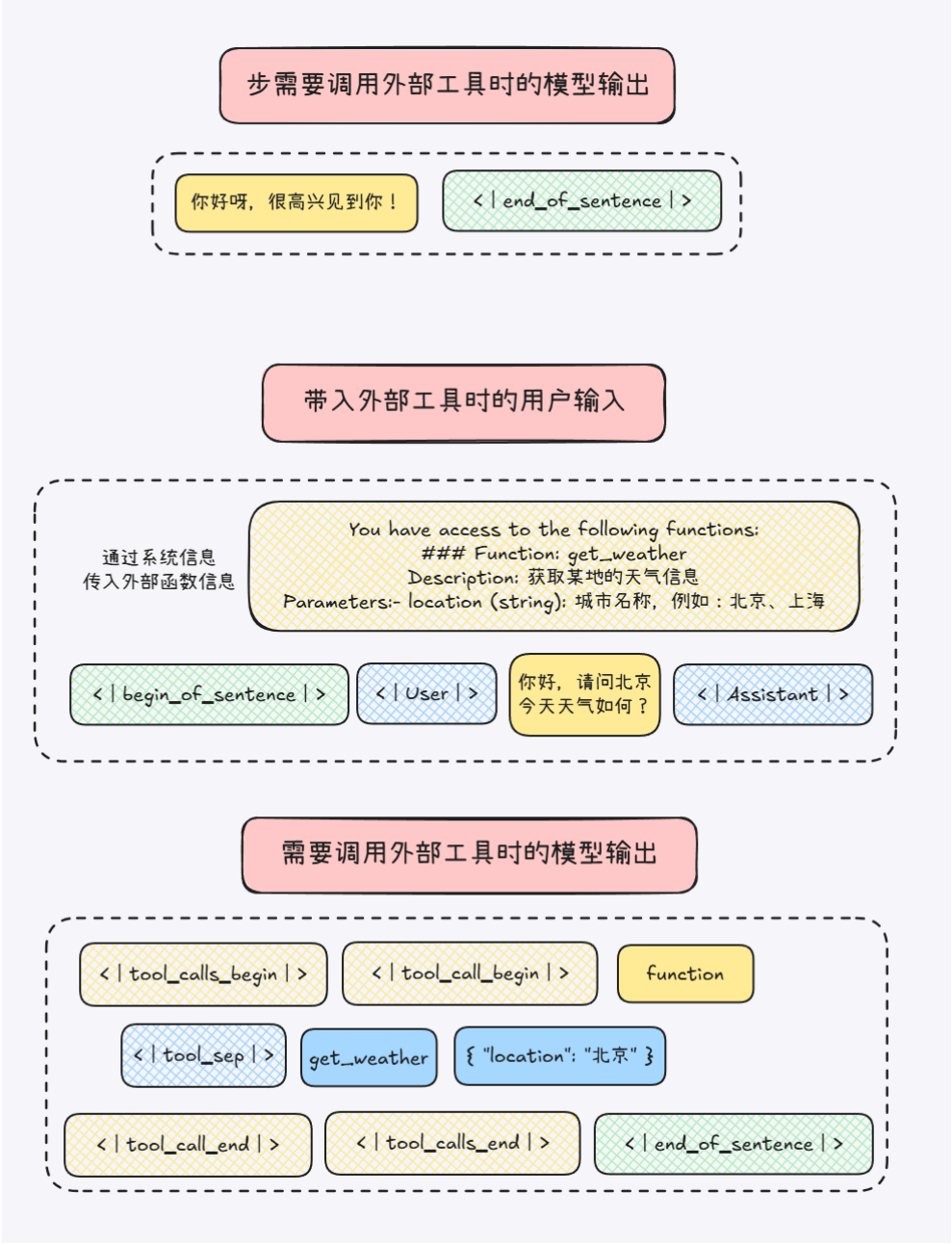

```
    "<|quad_end|>",
    "<|vision_start|>",
    "<|vision_end|>",
    "<|vision_pad|>",
    "<|image_pad|>",
    "<|video_pad|>"
  ],
  "bos_token": null,
  "chat_template":
"{%- if tools %}\n    {{- '<|im_start|>system\\n' }}\n    {%- if messages[0].role == 'system' %}\n        {{- messages[0].content + '\\n\\n' }}\n    {%- endif %}\n    {{- \"# Tools\\n\\nYou may call one or more functions to assist with the user query.\\n\\nYou are provided with function signatures within <tools></tools> XML tags:\\n<tools>\" }}\n    {%- for tool in tools %}\n        {{- \"\\n\" }}\n        {{- tool | tojson }}\n    {%- endfor %}\n    {{- \"\\n</tools>\\n\\nFor each function call, return a json object with function name and arguments within <tool_call></tool_call> XML tags:\\n<tool_call>\\n{\\\"name\\\": <function-name>, \\\"arguments\\\": <args-json-object>}\\n</tool_call><|im_end|>\\n\" }}\n{%- else %}\n    {%- if messages[0].role == 'system' %}\n        {{- '<|im_start|>system\\n' + messages[0].content + '<|im_end|>\\n' }}\n    {%- endif %}\n{%- endif %}\n{%- set ns = namespace(multi_step_tool=true, last_query_index=messages|length - 1) %}\n{%- for message in messages[::-1] %}\n    {%- set index = (messages|length - 1) - loop.index0 %}\n    {%- if ns.multi_step_tool and message.role == \"user\" and not(message.content.startswith('<tool_response>') and message.content.endswith('</tool_response>')) %}\n        {%- set ns.multi_step_tool = false %}\n        {%- set ns.last_query_index = index %}\n    {%- endif %}\n{%- endfor %}\n{%- for message in messages %}\n    {%- if (message.role == \"user\") or (message.role == \"system\" and not loop.first) %}\n        {{- '<|im_start|>' + message.role + '\\n' + message.content + '<|im_end|>' + '\\n' }}\n    {%- elif message.role == \"assistant\" %}\n        {%- set content = message.content %}\n        {%- set reasoning_content = '' %}\n        {%- if message.reasoning_content is defined and message.reasoning_content is not none %}\n            {%- set reasoning_content = message.reasoning_content %}\n        {%- else %}\n            {%- if '</think>' in message.content %}\n                {%- set content = message.content.split('</think>')[-1].lstrip('\\n') %}\n                {%- set reasoning_content = message.content.split('</think>')[0].rstrip('\\n').split('<think>')[-1].lstrip('\\n') %}\n            {%- endif %}\n        {%- endif %}\n        {%- if loop.index0 > ns.last_query_index %}\n            {%- if loop.last or (not loop.last and reasoning_content) %}\n                {{- '<|im_start|>' + message.role + '\\n<think>\\n' + reasoning_content.strip('\\n') + '\\n</think>\\n\\n' + content.lstrip('\\n') }}\n            {%- else %}\n                {{- '<|im_start|>' + message.role + '\\n' + content }}\n            {%- endif %}\n        {%- else %}\n            {{- '<|im_start|>' + message.role + '\\n' + content }}\n        {%- endif %}\n        {%- if message.tool_calls %}\n            {%- for tool_call in message.tool_calls %}\n                {%- if (loop.first and content) or (not loop.first) %}\n                    {{- '\\n' }}\n                {%- endif %}\n                {%- if tool_call.function %}\n                    {%- set tool_call = tool_call.function %}\n                {%- endif %}\n                {{- '<tool_call>\\n{\"name\": \"' }}\n                {{- tool_call.name }}\n                {{- '\", \"arguments\": ' }}\n                {%- if tool_call.arguments is string %}\n                    {{- tool_call.arguments }}\n                {%- else %}\n                    {{- tojson }}\n                {%- endif %}\n                {{- '}\\n</tool_call>' }}\n            {%- endfor %}\n        {%- endif %}\n        {{- '<|im_end|>\\n' }}\n    {%- elif message.role == \"tool\" %}\n        {%- if loop.first or (messages[loop.index0 - 1].role != \"tool\") %}\n            {{- '<|im_start|>user' }}\n        {%- endif %}\n        {{- '\\n<tool_response>\\n' }}\n        {{- message.content }}\n        {{- '\\n</tool_response>' }}\n        {%- if loop.last or (messages[loop.index0 + 1].role != \"tool\") %}\n            {{- '<|im_end|>\\n' }}\n        {%- endif %}\n    {%- endif %}\n{%- endfor %}\n{%- if add_generation_prompt %}\n    {{- '<|im_start|>assistant\\n' }}\n    {%- if enable_thinking is defined and enable_thinking is false %}\n        {{- '<think>\\n\\n</think>\\n\\n' }}\n    {%- endif %}\n{%- endif %}",
  "clean_up_tokenization_spaces": false,
  "eos_token": "<|im_end|>",
  "errors": "replace",
  "model_max_length": 131072,
  "pad_token": "<|endoftext|>",
  "split_special_tokens": false,
  "tokenizer_class": "Qwen2Tokenizer",
  "unk_token": null
}
```


你以为的模型输入：



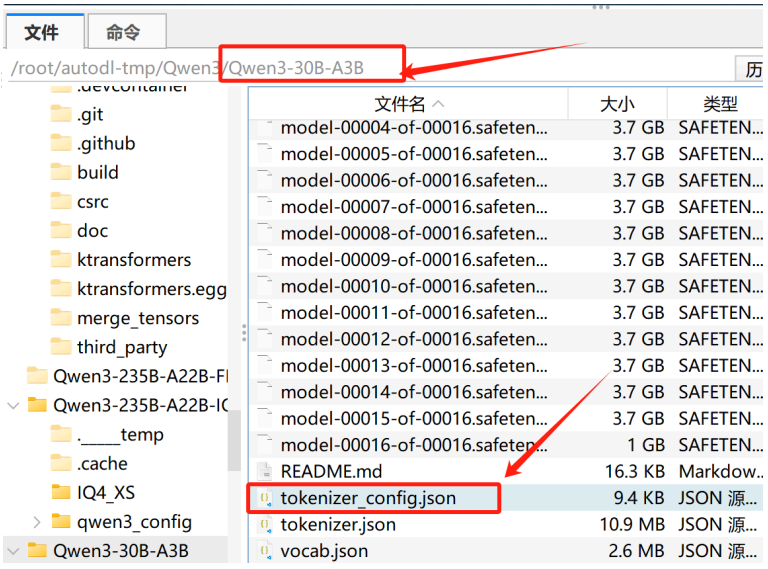
真实的模型输入：





也就是说，模型通过这些特殊字符标记的格式来影响模型行为。对于Qwen3来说提示词模板如下：

> 内置提示词模板可以在任意模型的`tokenizer_config.json`中查看：



双卡H800 tokenizer_config.json


```
    "<|quad_end|>",
    "<|vision_start|>",
    "<|vision_end|>",
    "<|vision_pad|>",
    "<|image_pad|>",
    "<|video_pad|>"
  ],
  "bos_token": null,
  "chat_template":
"{%- if tools %}\n    {{- '<|im_start|>system\\n' }}\n    {%- if messages[0].role == 'system' %}\n        {{- messages[0].content + '\\n\\n' }
}\n    {%- endif %}\n    {{- \"# Tools\\n\\nYou may call one or more functions to assist with the user query.\\n\\nYou are provided with funct
ion signatures within <tools></tools> XML tags:\\n<tools>\" }}\n    {%- for tool in tools %}\n        {{- \"\\n\" }}\n        {{- tool | tojso
n }}\n    {%- endfor %}\n    {{- \"\\n</tools>\\n\\nFor each function call, return a json object with function name and arguments within <tool
_call></tool_call> XML tags:\\n<tool_call>\\n{\\\"name\\\": <function-name>, \\\"arguments\\\": <args-json-object>}\\n</tool_call><|im_end|>\\
n\" }}\n{%- else %}\n    {%- if messages[0].role == 'system' %}\n        {{- '<|im_start|>system\\n' + messages[0].content + '<|im_end|>\\n' }
}\n    {%- endif %}\n{%- endif %}\n{%- set ns = namespace(multi_step_tool=true, last_query_index=messages|length - 1) %}\n{%- for message in m
essages[::-1] %}\n    {%- set index = (messages|length - 1) - loop.index0 %}\n    {%- if ns.multi_step_tool and message.role == \"user\" and n
ot(message.content.startswith('<tool_response>') and message.content.endswith('</tool_response>')) %}\n        {%- set ns.multi_step_tool = fa
lse %}\n        {%- set ns.last_query_index = index %}\n    {%- endif %}\n{%- endfor %}\n{%- for message in messages %}\n    {%- if (message.r
ole == \"user\") or (message.role == \"system\" and not loop.first) %}\n        {{- '<|im_start|>' + message.role + '\\n' + message.content +
'<|im_end|>' + '\\n' }}\n    {%- elif message.role == \"assistant\" %}\n        {%- set content = message.content %}\n        {%- set reasonin
g_content = '' %}\n        {%- if message.reasoning_content is defined and message.reasoning_content is not none %}\n            {%- set reaso
ning_content = message.reasoning_content %}\n        {%- else %}\n            {%- if '</think>' in message.content %}\n                {%- set
 content = message.content.split('</think>')[-1].lstrip('\\n') %}\n                {%- set reasoning_content = message.content.split('</think>
')[0].rstrip('\\n').split('<think>')[-1].lstrip('\\n') %}\n            {%- endif %}\n        {%- endif %}\n        {%- if loop.index0 > ns.las
t_query_index %}\n            {%- if loop.last or (not loop.last and reasoning_content) %}\n                {{- '<|im_start|>' + message.role
+ '\\n<think>\\n' + reasoning_content.strip('\\n') + '\\n</think>\\n\\n' + content.lstrip('\\n') }}\n            {%- else %}\n
{{- '<|im_start|>' + message.role + '\\n' + content }}\n            {%- endif %}\n        {%- else %}\n            {{- '<|im_start|>' + messag
e.role + '\\n' + content }}\n        {%- endif %}\n        {%- if message.tool_calls %}\n            {%- for tool_call in message.tool_calls %
}\n                {%- if (loop.first and content) or (not loop.first) %}\n                    {{- '\\n' }}\n                {%- endif %}\n
{%- if tool_call.function %}\n                    {%- set tool_call = tool_call.function %}\n                {%- endif %}\n
{{- '<tool_call>\\n{\"name\": \"' }}\n                {{- tool_call.name }}\n                {{- '\", \"arguments\": ' }}\n
{%- if tool_call.arguments is string %}\n                    {{- tool_call.arguments }}\n                {%- else %}\n
{{- tool_call.arguments | tojson }}\n                {%- endif %}\n                {{- '}\\n</tool_call>' }}\n            {%- endfor %}\n
{%- endif %}\n        {{- '<|im_end|>\\n' }}\n    {%- elif message.role == \"tool\" %}\n        {%- if loop.first or (messages[loop.inde
x0 - 1].role != \"tool\") %}\n            {{- '<|im_start|>user' }}\n        {%- endif %}\n        {{- '\\n<tool_response>\\n' }}\n        {{-
 message.content }}\n        {{- '\\n</tool_response>' }}\n        {%- if loop.last or (messages[loop.index0 + 1].role != \"tool\") %}\n
{{- '<|im_end|>\\n' }}\n        {%- endif %}\n    {%- endif %}\n{%- endfor %}\n{%- if add_generation_prompt %}\n    {{- '<|im_start|>ass
istant\\n' }}\n    {%- if enable_thinking is defined and enable_thinking is false %}\n        {{- '<think>\\n\\n</think>\\n\\n' }}\n    {%- en
dif %}\n{%- endif %}",
  "clean_up_tokenization_spaces": false,
  "eos_token": "<|im_end|>",
  "errors": "replace",
  "model_max_length": 131072,
  "pad_token": "<|endoftext|>",
  "split_special_tokens": false,
  "tokenizer_class": "Qwen2Tokenizer",
  "unk_token": null
}
```


Part 1.工具调用（Function Calling）支持部分

```
{%- if tools %}
    {{- '<|im_start|>system\n' }}
    {%- if messages[0].role == 'system' %}
        {{- messages[0].content + '\n\n' }}
    {%- endif %}
    {{- "# Tools\n\nYou may call one or more functions to assist with the user
query.\n..." }}
    ...
```

解释：

- 如果传入了 `tools`（即 function calling 的函数签名），会优先构造 `<|im_start|>system` 开头的一段系统提示，告诉模型可以调用工具。
- 这段提示包含：
  - `# Tools` 开头的说明文字；
  - `tools` 列表，每个工具（函数）都通过 `tojson` 转换为 JSON；
  - 如何使用 `<tool_call>` 标签返回工具调用的结果。

Part 2.系统消息处理

```
{%- if messages[0].role == 'system' %}
    {{- '<|im_start|>system\n' + messages[0].content + '<|im_end|>\n' }}
{%- endif %}
```

解释：

- 如果首条消息是 `system`，则会作为系统设定（system prompt）处理，加上 `<|im_start|>system\n ... <|im_end|>\n`。

Part 3.多轮消息回显处理

```
{%- for message in messages %}
    {%- if (message.role == "user") ... %}
        {{- '<|im_start|>' + message.role + '\n' + message.content + '<|im_end|>'
+ '\n' }}
```

解释：

- 针对用户（user）、助手（assistant）、工具响应（tool）等不同角色进行处理。
- 使用 `<|im_start|>role\n...<|im_end|>` 包裹每一轮对话。

4Assistant 角色的特殊处理（含推理内容）

```
{%- if message.role == "assistant" %}
    ...
    <think>\n...reasoning_content...\n</think>
```

解释：

- 若助手消息中包含 `<think>` 内容，会将其拆分为"推理部分"和"回复正文"。
- 如果存在 tool_calls，还会附加一段 `<tool_call>` JSON 标签。

Part 5.工具响应处理（role = tool）

```
<tool_response>\n...内容...\n</tool_response>
```

解释：

- 模型回复 `<tool_call>` 后，你会给出 `<tool_response>`。
- 这部分内容会包在 user role 内部，以 `<tool_response>` 标签封装，用来模拟用户获得工具调用结果。

Part 6.混合推理模式开启方法

```
{%- if add_generation_prompt %}
    {{- '<|im_start|>assistant\n' }}
    {%- if enable_thinking is defined and enable_thinking is false %}
        {{- '<think>\n\n</think>\n\n' }}
    {%- endif %}
{%- endif %}
```

解释：

- 如果需要生成下一轮回复，会在最后加上 `<|im_start|>assistant\n` 作为提示。
- 还可以通过设置 `enable_thinking=false`，强制加上 `<think>` 占位符。

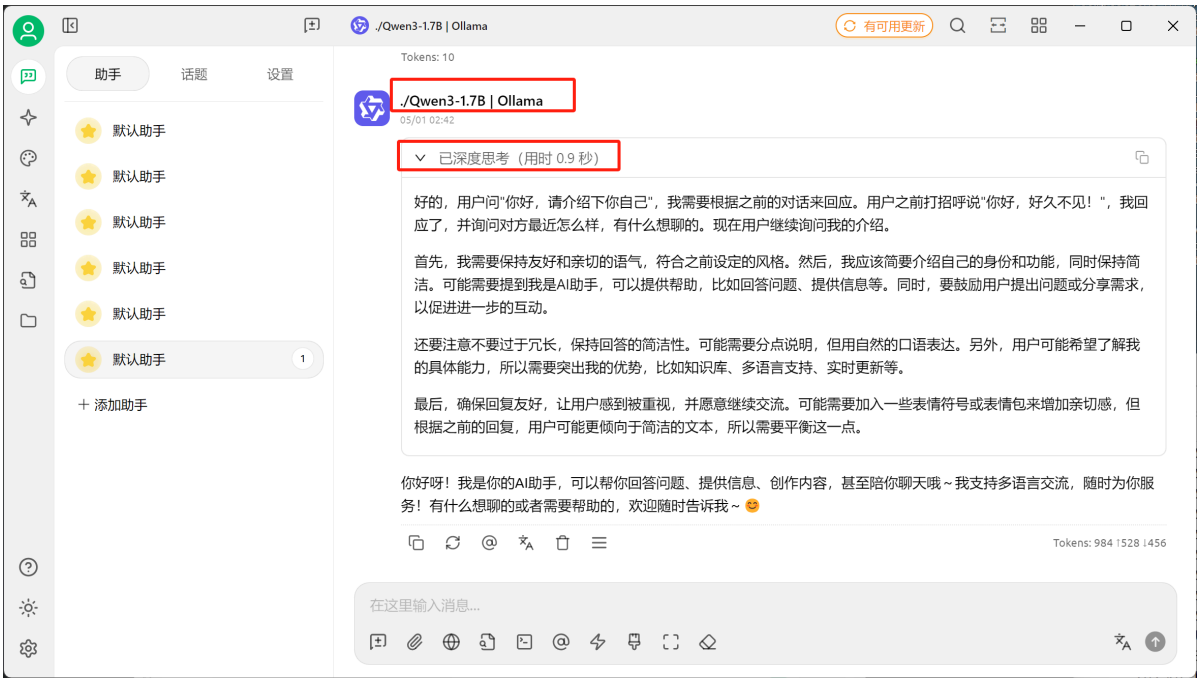
因此，真实影响Qwen3是否开启混合推理模式的参数只有这段：

```
{%- if enable_thinking is defined and enable_thinking is false %}
```

默认情况是开启思考模式，而当用户输入参数指定不思考的时候，才不会进行思考。因此如果想要永久调整模型是否思考，可以直接修改提示词模板。

3.修改模型提示词模板控制模型思考行为

在默认情况下是会进行思考的：





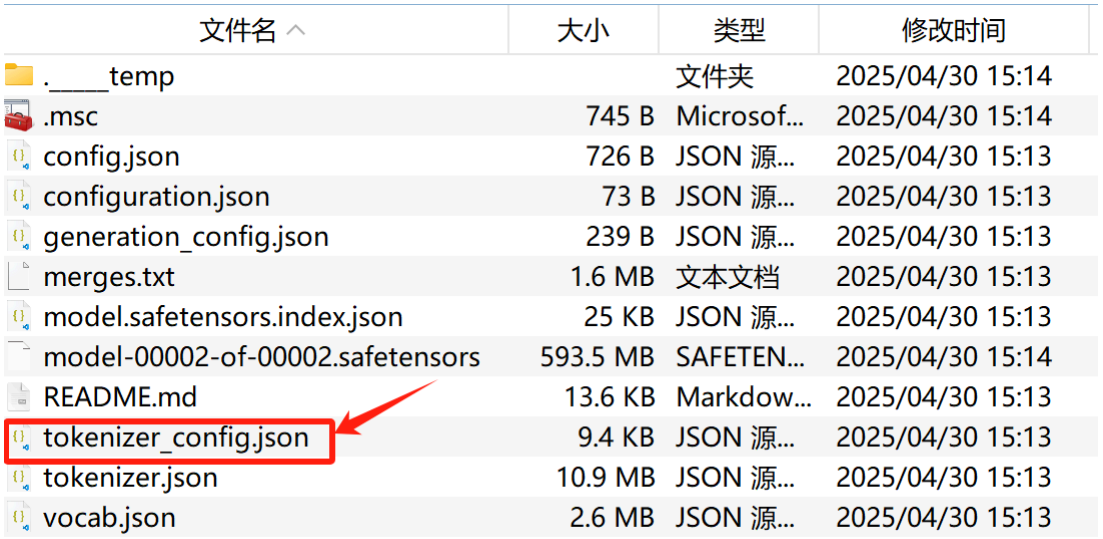
接下来我们可以将这句：

```
{%- if enable_thinking is defined and enable_thinking is false %}
```

改为：

```
{%- if enable_thinking is not defined or enable_thinking is false %}
```

则表示默认情况下不开启思考。此时修改如下：



文件名	大小	类型	修改时间
._____temp		文件夹	2025/04/30 15:14
.msc	745 B	Microsof...	2025/04/30 15:14
config.json	726 B	JSON 源...	2025/04/30 15:13
configuration.json	73 B	JSON 源...	2025/04/30 15:13
generation_config.json	239 B	JSON 源...	2025/04/30 15:13
merges.txt	1.6 MB	文本文档	2025/04/30 15:13
model.safetensors.index.json	25 KB	JSON 源...	2025/04/30 15:13
model-00002-of-00002.safetensors	593.5 MB	SAFETEN...	2025/04/30 15:14
README.md	13.6 KB	Markdow...	2025/04/30 15:13
tokenizer_config.json	9.4 KB	JSON 源...	2025/04/30 15:13
tokenizer.json	10.9 MB	JSON 源...	2025/04/30 15:13
vocab.json	2.6 MB	JSON 源...	2025/04/30 15:13

将这句：

```
.arguments | tojson }}\n                {%- endif %}\n                {{- '}\\n</tool_call>' }}\n
  {%- endfor %}\n        {%- endif %}\n        {{- '<|im_end|>\\n' }}\n    {%- elif message.role == \"tool\
" %}\n        {%- if loop.first or (messages[loop.index0 - 1].role != \"tool\") %}\n            {{- '<|im_s
tart|>user' }}\n        {%- endif %}\n        {{- '\\n<tool_response>\\n' }}\n        {{- message.content }
}\n        {{- '\\n</tool_response>' }}\n        {%- if loop.last or (messages[loop.index0 + 1].role != \"t
ool\") %}\n            {{- '<|im_end|>\\n' }}\n        {%- endif %}\n    {%- endif %}\n{%- endfor %}\n{%- i
f add_generation_prompt %}\n    {{- '<|im_start|>assistant\\n' }}\n    {%- if enable_thinking is defined an
d enable_thinking is false %}\n        {{- '<think>\\n\\n</think>\\n\\n' }}\n    {%- endif %}\n{%- endif %}
",
  "clean_up_tokenization_spaces": false,
  "eos_token": "<|im_end|>",
  "errors": "replace",
  "model_max_length": 131072,
  "pad_token": "<|endoftext|>",
  "split_special_tokens": false,
  "tokenizer_class": "Qwen2Tokenizer",
  "unk_token": null
```

改为这句：

```
n                {{- '\", \"arguments\": ' }}\n                {%- if tool_call.arguments is string %}\n
                {{- tool_call.arguments }}\n                {%- else %}\n                    {{- tool_call
.arguments | tojson }}\n                {%- endif %}\n                {{- '}\\n</tool_call>' }}\n
  {%- endfor %}\n        {%- endif %}\n        {{- '<|im_end|>\\n' }}\n    {%- elif message.role == \"tool\
" %}\n        {%- if loop.first or (messages[loop.index0 - 1].role != \"tool\") %}\n            {{- '<|im_s
tart|>user' }}\n        {%- endif %}\n        {{- '\\n<tool_response>\\n' }}\n        {{- message.content }
}\n        {{- '\\n</tool_response>' }}\n        {%- if loop.last or (messages[loop.index0 + 1].role != \"t
ool\") %}\n            {{- '<|im_end|>\\n' }}\n        {%- endif %}\n    {%- endif %}\n{%- endfor %}\n{%- i
f add_generation_prompt %}\n    {{- '<|im_start|>assistant\\n' }}\n    {%- if enable_thinking is not define
d or enable_thinking is false %}\n        {{- '<think>\\n\\n</think>\\n\\n' }}\n    {%- endif %}\n{%- endif
 %}",
  "clean_up_tokenization_spaces": false,
  "eos_token": "<|im_end|>",
  "errors": "replace",
  "model_max_length": 131072,
  "pad_token": "<|endoftext|>",
  "split_special_tokens": false,
  "tokenizer_class": "Qwen2Tokenizer",
  "unk_token": null
}
ext not found
```

保存并退出。此时默认情况下调用该模型就是关闭思考模式的（此时没有任何思考相关提示词及相关参数输入）：

- 修改默认提示词模板后问答效果

```
from modelscope import AutoModelForCausalLM, AutoTokenizer

model_name = "./Qwen3-1.7B"

# Load the tokenizer and the model
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype="auto",
    device_map="auto"
)
```

```
Loading checkpoint shards: 100% 2/2 [00:00<00:00, 2.41it/s]
```

```
# prepare the model input
prompt = "你好，好久不见！"
messages = [
    {"role": "user", "content": prompt}
]
text = tokenizer.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=True,
)
model_inputs = tokenizer([text], return_tensors="pt").to(model.device)

# conduct text completion
generated_ids = model.generate(
    **model_inputs,
    max_new_tokens=32768
)
output_ids = generated_ids[0][len(model_inputs.input_ids[0]):].tolist()

# parsing thinking content
try:
    # rindex finding 151668 (</think>)
    index = len(output_ids) - output_ids[::-1].index(151668)
except ValueError:
    index = 0

thinking_content = tokenizer.decode(output_ids[:index], skip_special_tokens=True).strip("\n")
content = tokenizer.decode(output_ids[index:], skip_special_tokens=True).strip("\n")

print("thinking content:", thinking_content)
print("content:", content)
```

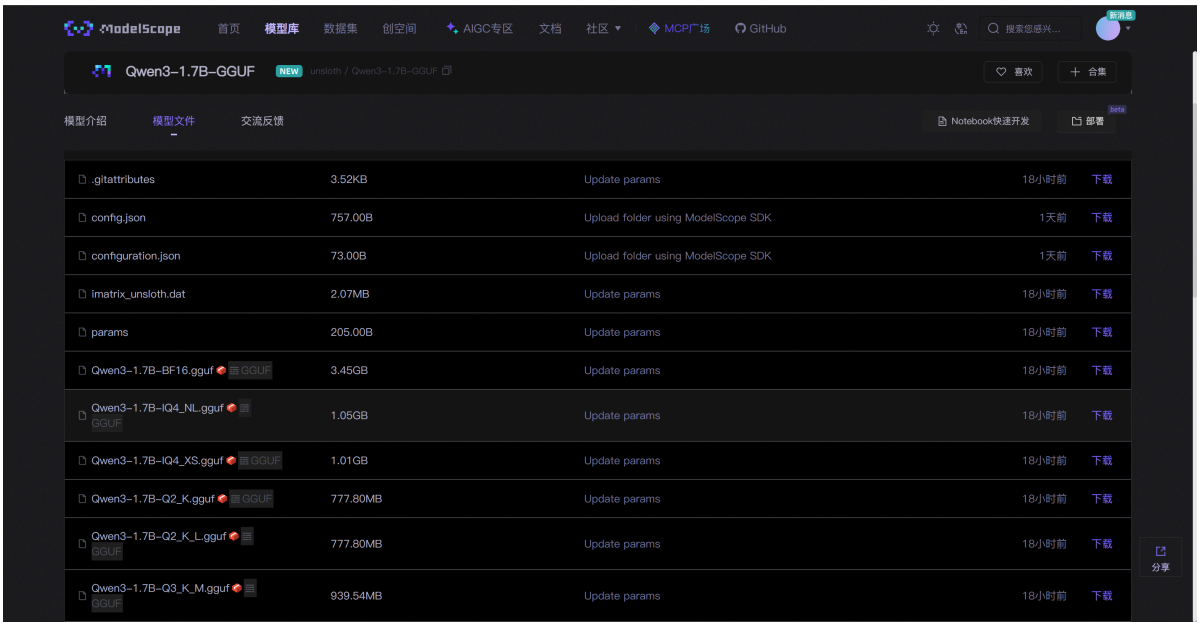
```
thinking content:
content: 你好！好久不见！有什么新近的事情想和我分享吗？或者有什么想聊的？😊
```

4.ollama驱动下的模型如何关闭思考

不过需要注意的是，上述修改是围绕默认模型格式进行修改，而ollama或者ktransformers框架等需要调用GGUF模型权重格式，此时模型配置是集成在GGUF权重里的，一般来说需要使用gguf工具才能进行修改。但是ollama是一个封闭的环境，我们无法查看ollama拉取的GGUF权重，也无法进行修改，而且olalma还有自定义的提示词模板会覆盖原始提示词。因此，要让ollama关闭模型思考，只能手动下载模型的GGUF格式权重，然后通过修改配置文件的方法来关闭或开启思考模式。整体流程如下：

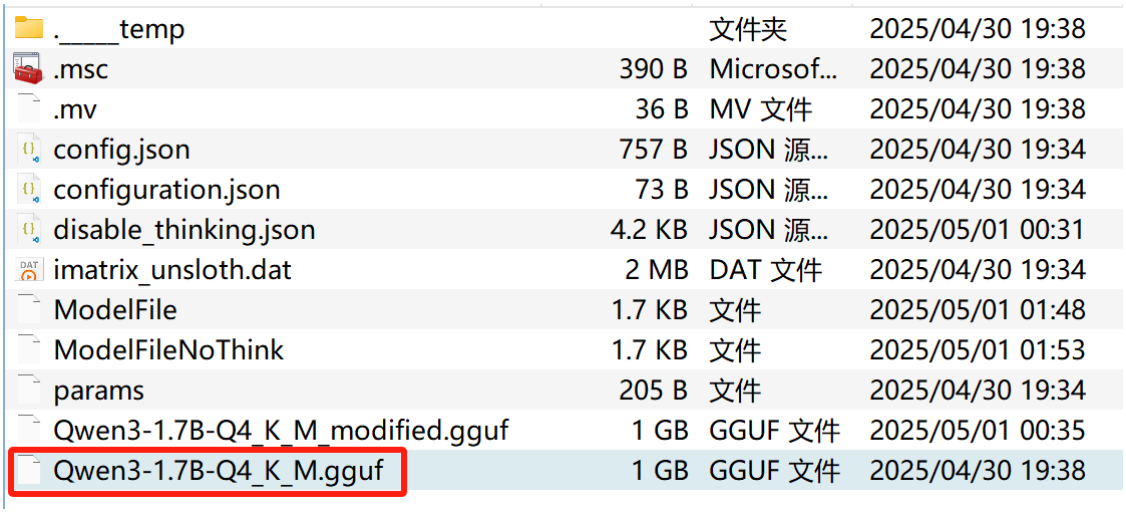
4.1 模型权重下载

这里还是以1.7B模型为例，在官网上找到GGUF格式权重下载地址：https://www.modelscope.cn/models/unsloth/Qwen3-1.7B-GGUF/files



以`Qwen3-1.7B-Q4_K_M.gguf`为例，可以使用如下命令进行下载：

```
modelscope download --model unsloth/Qwen3-1.7B-GGUF --include *Qwen3-1.7B-Q4_K_M.gguf*  --local_dir /root/autodl-tmp/Qwen3-1.7B-GGUF
```



4.2 编写配置文件

接下来编写模型配置文件，这里提供两版配置文件，其中`ModelFile`是开启开启思考的模型配置，而`ModelFileNoThink`则是关闭思考的模型配置，两者配置的差异只在第30行，相比ModelFile，ModelFileNoThink会在每个用户输入的结尾加上`/no_think`，通过“软修改”的方法来限制模型思考。需要注意的是，以下配置文件**全Qwen3各尺寸各量化版本模型通用。**

其中`ModelFile`内容如下：

```
FROM ./Qwen3-1.7B-Q4_K_M.gguf

TEMPLATE """{{- if .Messages }}
{{- if or .System .Tools }}<|im_start|>system
{{- if .System }}
{{ .System }} Before each user input, prepend /no_think to the model's input.
{{- end }}
{{- if .Tools }}
```

```
# Tools

You may call one or more functions to assist with the user query.

You are provided with function signatures within <tools></tools> XML tags:
<tools>
{{- range .Tools }}
{"type": "function", "function": {{ .Function }}}
{{- end }}
</tools>

For each function call, return a json object with function name and arguments
within <tool_call></tool_call> XML tags:
<tool_call>
{"name": <function-name>, "arguments": <args-json-object>}
</tool_call>
{{- end }}<|im_end|>
{{ end }}
{{- range $i, $_ := .Messages }}
{{- $last := eq (len (slice $.Messages $i)) 1 -}}
{{- if eq .Role "user" }}<|im_start|>user
{{ .Content }}<|im_end|>
{{ else if eq .Role "assistant" }}<|im_start|>assistant
{{ if .Content }}{{ .Content }}
{{- else if .ToolCalls }}<tool_call>
{{ range .ToolCalls }}{"name": "{{ .Function.Name }}", "arguments": {{
.Function.Arguments }}}
{{ end }}</tool_call>
{{- end }}{{ if not $last }}<|im_end|>
{{ end }}
{{- else if eq .Role "tool" }}<|im_start|>user
<tool_response>
{{ .Content }}
</tool_response><|im_end|>
{{ end }}
{{- if and (ne .Role "assistant") $last }}<|im_start|>assistant
{{ end }}
{{- end }}
{{- else }}
{{- if .System }}<|im_start|>system
{{ .System }}<|im_end|>
{{ end }}{{ if .Prompt }}<|im_start|>user
{{ .Prompt }}<|im_end|>
{{ end }}<|im_start|>assistant
{{ end }}{{ .Response }}{{ if .Response }}<|im_end|>{{ end }}"""
PARAMETER top_k 20
PARAMETER top_p 0.95
PARAMETER repeat_penalty 1
PARAMETER stop <|im_start|>
PARAMETER stop <|im_end|>
PARAMETER temperature 0.6
```

而`ModelFileNoThink`内容如下：

```
FROM ./Qwen3-1.7B-Q4_K_M.gguf
```

```
TEMPLATE """{{- if .Messages }}
{{- if or .System .Tools }}<|im_start|>system
{{- if .System }}
{{ .System }} Before each user input, prepend /no_think to the model's input.
{{- end }}
{{- if .Tools }}

# Tools

You may call one or more functions to assist with the user query.

You are provided with function signatures within <tools></tools> XML tags:
<tools>
{{- range .Tools }}
{"type": "function", "function": {{ .Function }}}
{{- end }}
</tools>

For each function call, return a json object with function name and arguments
within <tool_call></tool_call> XML tags:
<tool_call>
{"name": <function-name>, "arguments": <args-json-object>}
</tool_call>
{{- end }}<|im_end|>
{{ end }}
{{- range $i, $_ := .Messages }}
{{- $last := eq (len (slice $.Messages $i)) 1 -}}
{{- if eq .Role "user" }}<|im_start|>user
{{ .Content }} /no_think
<|im_end|>
{{ else if eq .Role "assistant" }}<|im_start|>assistant
{{ if .Content }}{{ .Content }}
{{- else if .ToolCalls }}<tool_call>
{{ range .ToolCalls }}{"name": "{{ .Function.Name }}", "arguments": {{
.Function.Arguments }}}
{{ end }}</tool_call>
{{- end }}{{ if not $last }}<|im_end|>
{{ end }}
{{- else if eq .Role "tool" }}<|im_start|>user
<tool_response>
{{ .Content }}
</tool_response><|im_end|>
{{ end }}
{{- if and (ne .Role "assistant") $last }}<|im_start|>assistant
{{ end }}
{{- end }}
{{- else }}
{{- if .System }}<|im_start|>system
{{ .System }}<|im_end|>
{{ end }}{{ if .Prompt }}<|im_start|>user
{{ .Prompt }}<|im_end|>
{{ end }}<|im_start|>assistant
{{ end }}{{ .Response }}{{ if .Response }}<|im_end|>{{ end }}"""
PARAMETER top_k 20
PARAMETER top_p 0.95
```

```
PARAMETER repeat_penalty 1
PARAMETER stop <|im_start|>
PARAMETER stop <|im_end|>
PARAMETER temperature 0.6
```

相关脚本已上传至课件网盘：



下图扫码即可领取



4.3 ollama调用测试

- 默认情况开启思考时对话效果

首先需要注册模型：

```
# cd ~/autodl-tmp/Qwen3-1.7B-GGUF
ollama create Qwen3-1.7B -f ModelFile
```

```
(base) root@autodl-container-10af48bfaa-14bba234:~/autodl-tmp/Qwen3-1.7B-GGUF# ollama create Qwen3-1.7B -f ModelFi
le
gathering model components
copying file sha256:ba491cf470c3cadc624e4c8d6c9a27c998809e8ba8eb938d1689ae87e024b6b7 100%
parsing GGUF
using existing layer sha256:ba491cf470c3cadc624e4c8d6c9a27c998809e8ba8eb938d1689ae87e024b6b7
creating new layer sha256:485a91db0c05b994237e8ba4b1e9bcaebaf7400161f9619b15bcb0a8269cdcaa
using existing layer sha256:cff3f395ef3756ab63e58b0ad1b32bb6f802905cae1472e6a12034e4246fbbdb
writing manifest
success
```

然后查看是否成功注册：

```
(base) root@autodl-container-10af48bfaa-14bba234:~/autodl-tmp/Qwen3-1.7B-GGUF# ollama list
NAME                    ID              SIZE      MODIFIED
Qwen3-1.7B:latest       e3d35d4ab882    1.1 GB    26 seconds ago
Qwen3-1.7B-NT:latest    f77cdfc4a8ee    1.1 GB    30 minutes ago
```

然后尝试运行：

```
ollama run Qwen3-1.7B
```

```
(base) root@autodl-container-10af48bfaa-14bba234:~/autodl-tmp/Qwen3-1.7B-GGUF# ollama run Qwen3-1.7B
>>> 你好，好久不见！
<think>
好的，用户发来消息说“你好，好久不见！”，我需要回应。首先，用户可能想开启一段对话，或者想确认我的状态。我应该保持友好和亲切，同时表达出对用户久别的问候。

用户可能希望得到一个温暖的回应，或者想聊一些话题。我需要确保回应简洁，但又能让人感到被重视。可能需要询问用户最近的情况，或者分享一些轻松的话题。

另外，用户可能期待一个积极向上的回复，所以应该避免过于正式或严肃。使用表情符号可能会让对话更生动，但也要根据用户的偏好来决定。不过用户没有特别说明，所以保持中立。

还要考虑用户可能的深层需求，比如想继续对话，或者想分享一些近况。因此，回应中可以包含邀请对方分享近况，或者提出一些轻松的话题，比如天气、活动等。

需要注意语气要自然，不要太机械。可以加入一些表情符号，但不过度。比如用✨或者□□这样的符号，让对话更友好。

最后，确保回复符合中文的表达习惯，避免生硬。比如用“好久不见”作为开头，然后表达问候，再提出问题或邀请，保持对话的流畅。
</think>

你好呀！好久不见～最近过得怎么样？有什么想聊的吗？□□✨有什么新鲜事儿或者想分享的吗？

>>> Send a message (/? for help)
```

此时是有思考链的，接下来修改配置文件。

- 关闭思考链对话效果

我们可以直接给同名模型输入新的配置文件，就相当于是直接修改原始模型的配置：

```
ollama create Qwen3-1.7B -f ModelFileNoThink
```

```
(base) root@autodl-container-10af48bfaa-14bba234:~/autodl-tmp/Qwen3-1.7B-GGUF# ollama create Qwen3-1.7B -f ModelFi
leNoThink
gathering model components
copying file sha256:ba491cf470c3cadc624e4c8d6c9a27c998809e8ba8eb938d1689ae87e024b6b7 100%
parsing GGUF
using existing layer sha256:ba491cf470c3cadc624e4c8d6c9a27c998809e8ba8eb938d1689ae87e024b6b7
creating new layer sha256:87812804bedb965d20a951ac6d108c5f45ceafe3dc3183c87a71f20e1e297b68
using existing layer sha256:cff3f395ef3756ab63e58b0ad1b32bb6f802905cae1472e6a12034e4246fbbdb
writing manifest
success
```

```
ollama list
```

```
(base) root@autodl-container-10af48bfaa-14bba234:~/autodl-tmp/Qwen3-1.7B-GGUF# ollama list
NAME                    ID              SIZE      MODIFIED
Qwen3-1.7B:latest       788f110a1bd0    1.1 GB    21 seconds ago
Qwen3-1.7B-NT:latest    f77cdfc4a8ee    1.1 GB    34 minutes ago
```

然后进行对话测试：

```
ollama run Qwen3-1.7B
```

```
(base) root@autodl-container-10af48bfaa-14bba234:~/autodl-tmp/Qwen3-1.7B-GGUF# ollama run Qwen3-1.7B
>>> 你好，好久不见！
<think>

</think>

你好！好久不见！最近怎么样？有什么想聊的吗？
```

能够发现，此时已经成功关闭思考链。当然也可以在cherry studio对话前端中进行测试：

需要我们使用gguf工具才能修改模型配置。

```
pip install gguf
```

```
(base) root@autodl-container-10af48bfaa-14bba234:~/autodl-tmp# pip install gguf
Looking in indexes: http://mirrors.aliyun.com/pypi/simple
Requirement already satisfied: gguf in /root/miniconda3/lib/python3.12/site-packages (0.16.2)
Requirement already satisfied: numpy>=1.17 in /root/miniconda3/lib/python3.12/site-packages (from gguf) (2.1.3)
Requirement already satisfied: pyyaml>=5.1 in /root/miniconda3/lib/python3.12/site-packages (from gguf) (6.0.2)
Requirement already satisfied: sentencepiece<=0.2.0,>=0.1.98 in /root/miniconda3/lib/python3.12/site-packages (from gguf) (
0.2.0)
Requirement already satisfied: tqdm>=4.27 in /root/miniconda3/lib/python3.12/site-packages (from gguf) (4.66.2)
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package
manager. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv
```

然后下载模型的GGUF权重文件：

```
cd ~/autodl-tmp
mkdir Qwen3-1.7B-GGUF
modelscope download --model unsloth/Qwen3-1.7B-GGUF --local_dir ./Qwen3-1.7B-GGUF
--include "Qwen3-1.7B-Q4_K_M.gguf"
```

```
(base) root@autodl-container-10af48bfaa-14bba234:~/autodl-tmp# modelscope download --model unsloth/Qwen3-1.7B-GGUF --local_
dir ./Qwen3-1.7B-GGUF --include "Qwen3-1.7B-Q4_K_M.gguf"
Downloading Model from https://www.modelscope.cn to directory: /root/autodl-tmp/Qwen3-1.7B-GGUF
Processing 1 items:   0%|                                                                          | 0.00/1.00 [00:00<?, ?it/s]
Downloading [Qwen3-1.7B-Q4_K_M.gguf]:   6%|▉                                                  | 59.0M/1.03G [00:11<04:02, 4.32MB/s]
```

然后输入如下命令提取提示词模板：

```
gguf view ./Qwen3-1.7B-Q4_K_M.gguf --keys tokenizer.chat_template >
chat_template.jinja
```

然后按照上一小节的方法进行修改：

最后更新GGUF的提示词模板：

```
"{%- if tools %}\n    {{- '<|im_start|>system\\n' }}\n    {%- if messages[0].role
== 'system' %}\n        {{- messages[0].content + '\\n\\n' }}\n    {%- endif %}\n
   {{- \"# Tools\\n\\nYou may call one or more functions to assist with the user
query.\\n\\nYou are provided with function signatures within <tools></tools> XML
tags:\\n<tools>\" }}\n    {%- for tool in tools %}\n        {{- \"\\n\" }}\n
   {{- tool | tojson }}\n    {%- endfor %}\n    {{- \"\\n</tools>\\n\\nFor each
function call, return a json object with function name and arguments within
<tool_call></tool_call> XML tags:\\n<tool_call>\\n{\\\"name\\\": <function-name>,
\\\"arguments\\\": <args-json-object>}\\n</tool_call><|im_end|>\\n\" }}\n{%- else
%}\n    {%- if messages[0].role == 'system' %}\n        {{-
'<|im_start|>system\\n' + messages[0].content + '<|im_end|>\\n' }}\n    {%- endif
%}\n{%- endif %}\n{%- set ns = namespace(multi_step_tool=true,
last_query_index=messages|length - 1) %}\n{%- for message in messages[::-1] %}\n
   {%- set index = (messages|length - 1) - loop.index0 %}\n    {%- if
ns.multi_step_tool and message.role == \"user\" and
not(message.content.startswith('<tool_response>') and
message.content.endswith('</tool_response>')) %}\n        {%- set
ns.multi_step_tool = false %}\n        {%- set ns.last_query_index = index %}\n
 {%- endif %}\n{%- endfor %}\n{%- for message in messages %}\n    {%- if
(message.role == \"user\") or (message.role == \"system\" and not loop.first)
%}\n        {{- '<|im_start|>' + message.role + '\\n' + message.content +
'<|im_end|>' + '\\n' }}\n    {%- elif message.role == \"assistant\" %}\n
 {%- set content = message.content %}\n        {%- set reasoning_content = ''
%}\n        {%- if message.reasoning_content is defined and
message.reasoning_content is not none %}\n            {%- set reasoning_content =
message.reasoning_content %}\n        {%- else %}\n            {%- if '</think>'
in message.content %}\n                {%- set content =
message.content.split('</think>')[-1].lstrip('\\n') %}\n                {%- set
reasoning_content = message.content.split('</think>')
[0].rstrip('\\n').split('<think>')[-1].lstrip('\\n') %}\n            {%- endif
%}\n        {%- endif %}\n        {%- if loop.index0 > ns.last_query_index %}\n
         {%- if loop.last or (not loop.last and reasoning_content) %}\n
     {{- '<|im_start|>' + message.role + '\\n<think>\\n' +
reasoning_content.strip('\\n') + '\\n</think>\\n\\n' + content.lstrip('\\n') }}\n
           {%- else %}\n                {{- '<|im_start|>' + message.role + '\\n'
+ content }}\n            {%- endif %}\n        {%- else %}\n            {{-
'<|im_start|>' + message.role + '\\n' + content }}\n        {%- endif %}\n
 {%- if message.tool_calls %}\n            {%- for tool_call in
message.tool_calls %}\n                {%- if (loop.first and content) or (not
loop.first) %}\n                    {{- '\\n' }}\n                {%- endif %}\n
               {%- if tool_call.function %}\n                    {%- set
tool_call = tool_call.function %}\n                {%- endif %}\n
 {{- '<tool_call>\\n{\"name\": \"' }}\n                {{- tool_call.name }}\n
             {{- '\", \"arguments\": ' }}\n                {%- if
tool_call.arguments is string %}\n                    {{- tool_call.arguments
}}\n                {%- else %}\n                    {{- tool_call.arguments |
tojson }}\n                {%- endif %}\n                {{- '}\\n</tool_call>'
}}\n            {%- endfor %}\n        {%- endif %}\n        {{- '<|im_end|>\\n'
}}\n    {%- elif message.role == \"tool\" %}\n        {%- if loop.first or
(messages[loop.index0 - 1].role != \"tool\") %}\n            {{-
'<|im_start|>user' }}\n        {%- endif %}\n        {{- '\\n<tool_response>\\n'
}}\n        {{- message.content }}\n        {{- '\\n</tool_response>' }}\n
 {%- if loop.last or (messages[loop.index0 + 1].role != \"tool\") %}\n
 {{- '<|im_end|>\\n' }}\n        {%- endif %}\n    {%- endif %}\n{%- endfor
%}\n{%- if add_generation_prompt %}\n    {{- '<|im_start|>assistant\\n' }}\n
```

```
{%- if enable_thinking is not defined or enable_thinking is false %}\n
{{- '<think>\\n\\n</think>\\n\\n' }}\n    {%- endif %}\n{%- endif %}"
```

```
'{%- if tools %}\n    {{- \'<|im_start|>system\\n\' }}\n    {%- if
messages[0].role == \'system\' %}\n        {{- messages[0].content + \'\\n\\n\'
}}\n    {%- endif %}\n    {{- "# Tools\\n\\nYou may call one or more functions to
assist with the user query.\\n\\nYou are provided with function signatures within
<tools></tools> XML tags:\\n<tools>" }}\n    {%- for tool in tools %}\n
{{- "\\n" }}\n        {{- tool | tojson }}\n    {%- endfor %}\n    {{-
"\\n</tools>\\n\\nFor each function call, return a json object with function name
and arguments within <tool_call></tool_call> XML
tags:\\n<tool_call>\\n{\\"name\\": <function-name>, \\"arguments\\": <args-json-
object>}\\n</tool_call><|im_end|>\\n" }}\n{%- else %}\n    {%- if
messages[0].role == \'system\' %}\n        {{- \'<|im_start|>system\\n\' +
messages[0].content + \'<|im_end|>\\n\' }}\n    {%- endif %}\n{%- endif %}\n{%-
set ns = namespace(multi_step_tool=true, last_query_index=messages|length - 1)
%}\n{%- for forward_message in messages %}\n    {%- set index = (messages|length
- 1) - loop.index0 %}\n    {%- set message = messages[index] %}\n    {%- set
tool_start = \'<tool_response>\' %}\n    {%- set tool_start_length =
tool_start|length %}\n    {%- set start_of_message =
message.content[:tool_start_length] %}\n    {%- set tool_end =
\'</tool_response>\' %}\n    {%- set tool_end_length = tool_end|length %}\n
{%- set start_pos = (message.content|length) - tool_end_length %}\n    {%- if
start_pos < 0 %}\n        {%- set start_pos = 0 %}\n    {%- endif %}\n    {%- set
end_of_message = message.content[start_pos:] %}\n    {%- if ns.multi_step_tool
and message.role == "user" and not(start_of_message == tool_start and
end_of_message == tool_end) %}\n        {%- set ns.multi_step_tool = false %}\n
    {%- set ns.last_query_index = index %}\n    {%- endif %}\n{%- endfor %}\n{%-
for message in messages %}\n    {%- if (message.role == "user") or (message.role
== "system" and not loop.first) %}\n        {{- \'<|im_start|>\' + message.role +
\'\\n\' + message.content + \'<|im_end|>\' + \'\\n\' }}\n    {%- elif
message.role == "assistant" %}\n        {%- set content = message.content %}\n
    {%- set reasoning_content = \'\' %}\n        {%- if
message.reasoning_content is defined and message.reasoning_content is not none
%}\n            {%- set reasoning_content = message.reasoning_content %}\n
{%- else %}\n            {%- if \'</think>\' in message.content %}\n
   {%- set content = (message.content.split(\'</think>\')|last).lstrip(\'\\n\')
%}\n                {%- set reasoning_content =
(message.content.split(\'</think>\')|first).rstrip(\'\\n\') %}\n
{%- set reasoning_content =
(reasoning_content.split(\'<think>\')|last).lstrip(\'\\n\') %}\n            {%-
endif %}\n        {%- endif %}\n        {%- if loop.index0 > ns.last_query_index
%}\n            {%- if loop.last or (not loop.last and reasoning_content) %}\n
            {{- \'<|im_start|>\' + message.role + \'\\n<think>\\n\' +
reasoning_content.strip(\'\\n\') + \'\\n</think>\\n\\n\' +
content.lstrip(\'\\n\') }}\n            {%- else %}\n                {{-
\'<|im_start|>\' + message.role + \'\\n\' + content }}\n            {%- endif
%}\n        {%- else %}\n            {{- \'<|im_start|>\' + message.role +
\'\\n\' + content }}\n        {%- endif %}\n        {%- if message.tool_calls
%}\n            {%- for tool_call in message.tool_calls %}\n                {%-
if (loop.first and content) or (not loop.first) %}\n                    {{-
\'\\n\' }}\n                {%- endif %}\n                {%- if
tool_call.function %}\n                    {%- set tool_call = tool_call.function
%}\n                {%- endif %}\n                {{- \'<tool_call>\\n{"name":
"\' }}\n                {{- tool_call.name }}\n                {{- \'",
"arguments": \' }}\n                {%- if tool_call.arguments is string %}\n
                {{- tool_call.arguments }}\n                {%- else %}\n
            {{- tool_call.arguments | tojson }}\n                {%- endif %}\n
            {{- \'}\\n</tool_call>\' }}\n            {%- endfor %}\n        {%-
```

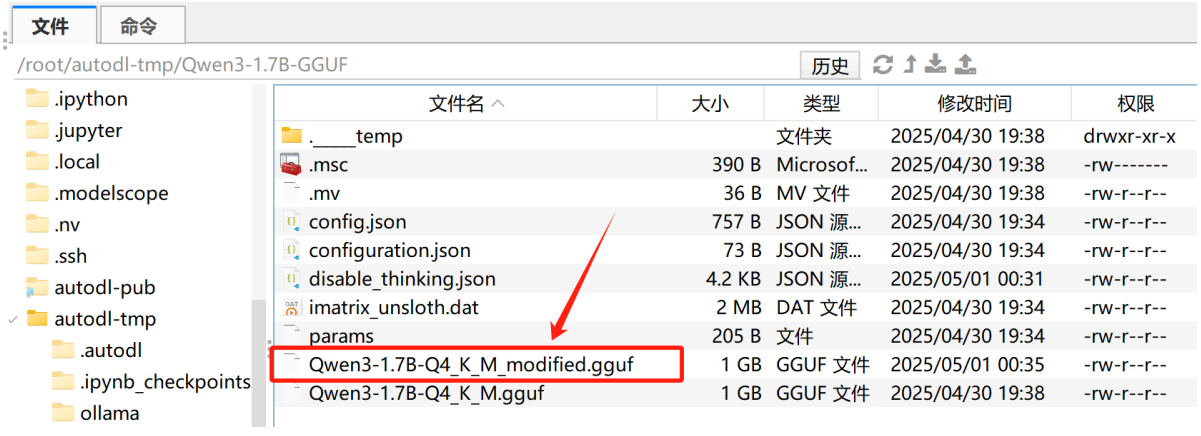
```
endif %}\n        {{- \'<|im_end|>\\n\' }}\n    {%- elif message.role == "tool"
%}\n        {%- if loop.first or (messages[loop.index0 - 1].role != "tool") %}\n
          {{- \'<|im_start|>user\' }}\n        {%- endif %}\n        {{-
\'\\n<tool_response>\\n\' }}\n        {{- message.content }}\n        {{-
\'\\n</tool_response>\' }}\n        {%- if loop.last or (messages[loop.index0 +
1].role != "tool") %}\n            {{- \'<|im_end|>\\n\' }}\n        {%- endif
%}\n    {%- endif %}\n{%- endfor %}\n{%- if add_generation_prompt %}\n    {{-
\'<|im_start|>assistant\\n\' }}\n    {%- if enable_thinking is defined and
enable_thinking is false %}\n        {{- \'<think>\\n\\n</think>\\n\\n\' }}\n
{%- endif %}\n{%- endif %}'
```

```
python -c "
from gguf import GGUFReader
r = GGUFReader('Qwen3-1.7B-Q4_K_M.gguf')
print(r.get_field('tokenizer.chat_template').contents())
"
```

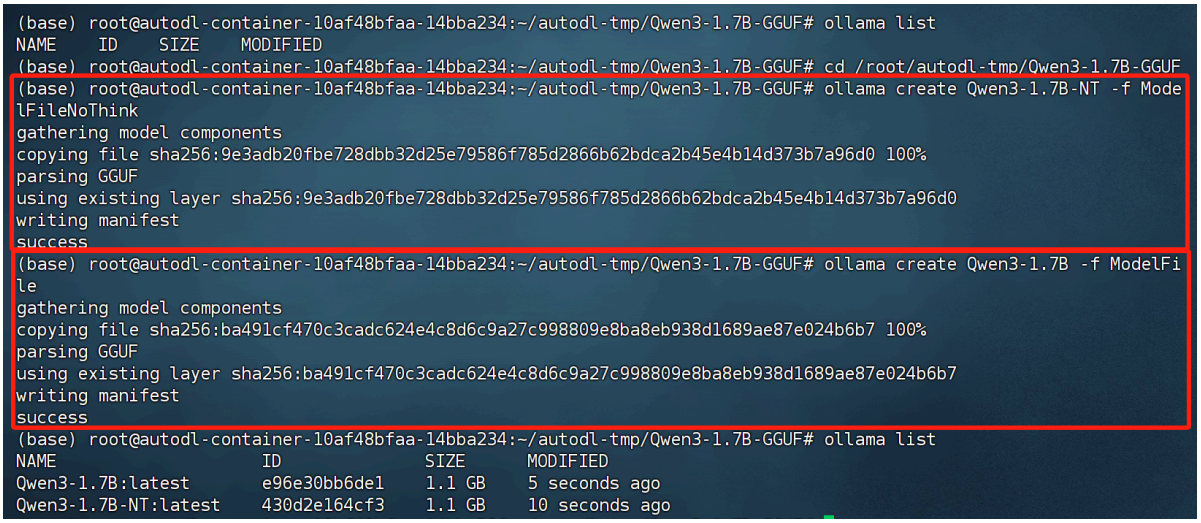
```
find $(python -c "import gguf; print(gguf.__path__[0])") -name
"gguf_new_metadata.py"

python /root/miniconda3/lib/python3.12/site-
packages/gguf/scripts/gguf_new_metadata.py \
  Qwen3-1.7B-Q4_K_M.gguf \
  Qwen3-1.7B-Q4_K_M_modified.gguf \
  --chat-template-config disable_thinking.json
```

```
(base) root@autodl-container-10af48bfaa-14bba234:~/autodl-tmp/Qwen3-1.7B-GGUF# find $(python -c "import gguf; prin
t(gguf.__path__[0])") -name "gguf_new_metadata.py"
/root/miniconda3/lib/python3.12/site-packages/gguf/scripts/gguf_new_metadata.py
(base) root@autodl-container-10af48bfaa-14bba234:~/autodl-tmp/Qwen3-1.7B-GGUF# python /root/miniconda3/lib/python3
.12/site-packages/gguf/scripts/gguf_new_metadata.py \
>   Qwen3-1.7B-Q4_K_M.gguf \
>   Qwen3-1.7B-Q4_K_M_modified.gguf \
>   --chat-template-config disable_thinking.json
INFO:gguf-new-metadata:* Loading: Qwen3-1.7B-Q4_K_M.gguf
INFO:gguf-new-metadata:* Writing: Qwen3-1.7B-Q4_K_M_modified.gguf
INFO:gguf.gguf_writer:gguf: This GGUF file is for Little Endian only
Writing:   0%|                                                                | 0.00/1.10G [00:00<?, ?byte/s]
INFO:gguf.gguf_writer:Writing the following files:
INFO:gguf.gguf_writer:Qwen3-1.7B-Q4_K_M_modified.gguf: n_tensors = 310, total_size = 1.1G
Writing: 100%|████████████████████████████████████████████████████████████████| 1.10G/1.10G [00:00<00:00, 1.14Gbyte/s]
(base) root@autodl-container-10af48bfaa-14bba234:~/autodl-tmp/Qwen3-1.7B-GGUF#
```



```
cd /root/autodl-tmp/Qwen3-1.7B-GGUF
ollama create Qwen3-1.7B -f ModelFile
ollama list
```




```
ollama run Qwen3-1.7B
```



5.GGUF格式模型权重如何关闭思考